

图 9.26 作为蓝牙主装置的个人电脑，HC-06 蓝牙从模块和 RS-232 控制器之间的连接设置

HC-06 蓝牙模块的默认设置如下：

波特率	9600
协议	N81（没有校验位，8 位数据，1 位停止位）
配对名	linvor
密码	1234

该模块可以通过设置控制面板中的设备和打印机，很容易地匹配到一个 Windows 个人电脑。选择添加设备。等一会儿，你的蓝牙设备（使用默认名称 ”linvor” ）将会出现。选择你的设备并继续。当需要密码时，输入默认密码 1234。你的蓝牙设备就会显示在列表中。

在个人电脑上，可以使用终端模拟器，比如 PuTTY，与蓝牙通信模块。在 PuTTY 配置界面，选择串行连接类型并输入正确的 COM 端口和传输速度。PuTTY 将字符发送到 RS-232 接收控制器的 8 位数据输出线上，同时，RS-232 发送方控制器将数据发到 PuTTY 中，这些数据将显示在 PuTTY 的终端屏幕上。

9.6 液晶显示控制器

16×2 的液晶显示器（LCD）如图 9.28 所示，是一个很常见的显示模块，它可以很容易地连接到微控制器。在这一节中，我们将介绍它的基本操作，并设计一个专门的微处理器来控制和显示一些示例字符。本节的讨论不包括模块的全部操作。该液晶显示模块完整和详细的描述和操作可以查阅制造商提供的数据手册。



图 9.2 一块 16×2 液晶显示器

9.6.1 操作原理

16×2 的 LCD 可以显示 2 行,每行 16 个字符，每个字符由一个 5×7 点阵格式构成。显示器内部有一个集成电路控制其操作，如如何控制每个字符矩阵的每个点打开和关闭以形成各种不同的字符，在哪里显示字符，是否显示光标，是否移位字符等功能。因此，将 LCD 连接到微处理器的本质是理解与内部集成电路正确联系所需的命令。除了电源线和地线，LCD 有三个控制信号（RS, R/W 和使能信号），还有八位或四位数据线。如果只用四位数据线，则只有高四位（D7～D4）连接，每个字节将被作为两个单独的十六进制数一个接一个地发出，其中最重要的十六进制数字先发送。RS 信号用来确定随后的数据位被解释为命令还是数据。如果 RS= 0，则这一位是一个命令，如果 RS= 1，则这一位是数据。R/W 信号确定数据读或写，0 = 写，1 =读。在使能信号的下降沿，RS, R/W 和数据信号被锁存到内部寄存器。

LCD 和专用微处理器之间的物理连接如图 9.29 所示。正如图中所看到的那样，三个控制信号和八个数据线的连接是非常直观的。如果选择使用 4 位数据线，则低四位（D3-D0）不用连接。现在的问题就是设计一个专用微处理器，来正确地联系 LCD，并发送必要的命令给它。

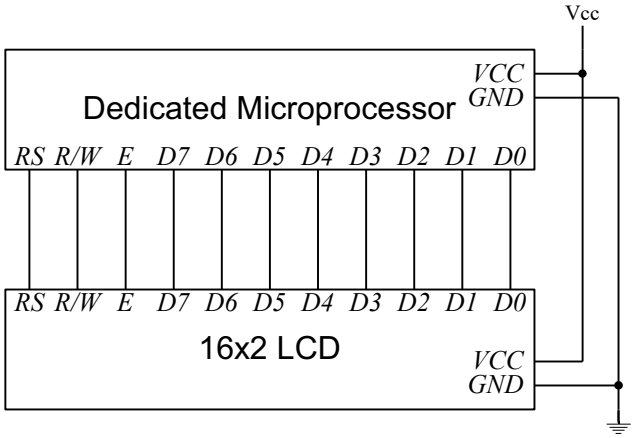


图 9.29 LCD 和专用微处理器之间的连接

在上电时，LCD 需要以一系列的命令来初始化，如图 9.30 所示。这个初始化过程包括一个十六进制代码的序列，用来告知 LCD：4 位或 8 位数据是否将被使用，LCD 的尺寸格式，将光标、增量模式、移动和闪烁等打开或关闭，然后清空显示屏。最初的十六进制代码 38x 被发送两次或更多次，以确保 LCD 模块成功进入 4 位或 8 位数据模式。然后用 0Ex 设置显示屏，06x 设置输入模式，最后用 01x 来清空显示屏。

在初始化序列之后，另一个给光标定位的命令被发送到 LCD，然后将所显示的字符发送到 LCD。如代码中所示，显示了“HELLO WORLD”。

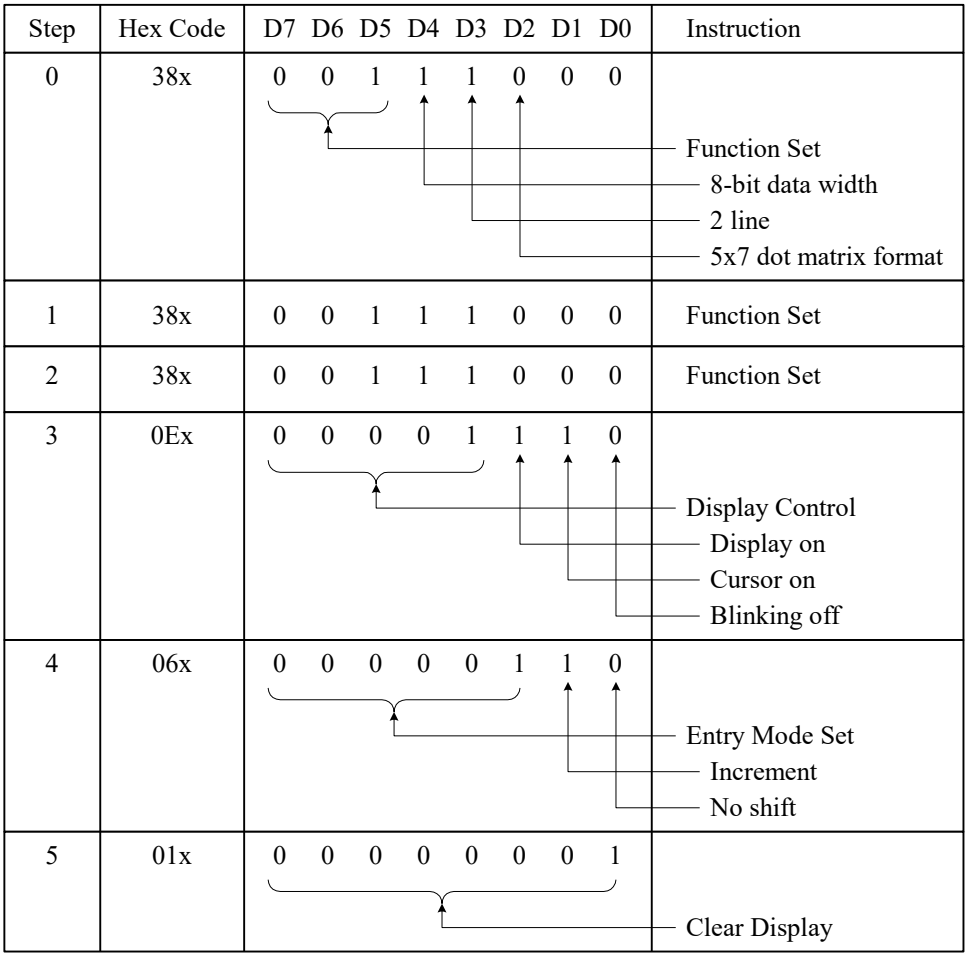


图 9.30 LCD 的初始化命令序列

9.6.2 控制器设计

控制 LCD 的专用单片机不需要数据通路。用 Verilog 和 VHDL 代码编写的 FSM 在图 9.31 和图 9.32 分别描述。FSM 状态的初始化完全按照图 9.30 所示的初始化步骤。每个十六进制码发送需要两个状态，因为在使能信号的下降沿 RS, R/W 和数据信号被锁存，所以我们每次都需要切换使能信号的状态。

```
module lcd_controller (
  input Clock, Reset,
  output reg [7:0] LCD_DATA,
  output reg LCD_RW,LCD_EN, LCD_RS,
  output LCD_ON, LCD_BLON
    );

  reg [3:0] state;
  reg [7:0] initcode[0:5];
  reg [7:0] line1[0:15];
  integer count;

  initial begin
    // LCD initialization sequence codes
```

```
  initcode[0] = 8'h38;   // Init three times
  initcode[1] = 8'h38;
  initcode[2] = 8'h38;
  initcode[3] = 8'h0E; //Display control: Display ON; Cursor ON; Blink OFF
  initcode[4] = 8'h06;   // Entry mode set: Increment One; No Shift
  initcode[5] = 8'h01;   // Display clear
  // HELLO  WORLD
  line1[0] = 8'h20;
  line1[1] = 8'h20;
  line1[2] = 8'h48;
  line1[3] = 8'h45;
  line1[4] = 8'h4C;
  line1[5] = 8'h4C;
  line1[6] = 8'h4F;
  line1[7] = 8'h20;
  line1[8] = 8'h20;
  line1[9] = 8'h57;
  line1[10] = 8'h4F;
  line1[11] = 8'h52;
  line1[12] = 8'h4C;
  line1[13] = 8'h44;
  line1[14] = 8'h20;
  line1[15] = 8'h20;
end

assign LCD_ON = 1'b1;
assign LCD_BLON = 1'b0;

always @ (posedge Clock, posedge Reset) begin
  if (Reset) begin
    count <= 0;
    state <= 4'b0000;
    end
  else
    case (state)
    // LCD initialization sequence
    // The LCD_DATA is written to the LCD at the falling edge of the E line
    // therefore we need to toggle the E line for each data write
    4'b0000: begin
      LCD_DATA <= initcode[count];
      LCD_EN <= 1'b1;  // EN=1;
      LCD_RS <= 1'b0;  // RS=0; an instruction
      LCD_RW <= 1'b0;  // R/W'=0; write
      state <= 4'b0001;
      end
    4'b0001: begin
      LCD_EN <= 1'b0;  // set EN=0;
      count <= count + 1;
      if (count+1 < 6)
        state <= 4'b0000;
      else
        state <= 4'b0010;
      end
    // move cursor to first line of display
    4'b0010: begin
      LCD_DATA <= 8'h80;  // x80 is address of 1st position on first line
    //LCD_DATA <= 8'hBF;  // xBF is address of 1st position on second line
```

```
      LCD_EN <= 1'b1;  // EN=1;
      LCD_RS <= 1'b0;  // RS=0; an instruction
      LCD_RW <= 1'b0;  // R/W'=0; write
      state <= 4'b0011;
      end
    4'b0011: begin
      LCD_EN <= 1'b0;  // EN=0; toggle EN
      count <= 0;
      state <= 4'b0100;
      end
    // write 1st line text
    4'b0100: begin
      LCD_DATA <= line1[count];
      LCD_EN <= 1'b1;  // EN=1;
      LCD_RS <= 1'b1;  // RS=1; data
      LCD_RW <= 1'b0;  // R/W'=0; write
      state <= 4'b0101;
      end
    4'b0101: begin
      LCD_EN <= 1'b0;  // EN=0; toggle EN
      count <= count + 1;
      if (count+1 < 16)
        state <= 4'b0100;
      else
        state <= 4'b1010;
      end
    4'b1010: begin
      state <= 4'b1010;
      end
    default: begin
      state <= 4'b1010;
      end
    endcase
  end // always
endmodule
```

图 9.31 16×2 LCD 控制器的 Verilog 代码

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY lcd_controller IS
PORT (
  Clock, Reset: IN STD_LOGIC;
  LCD_DATA: OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
  LCD_RW, LCD_EN, LCD_RS: OUT STD_LOGIC;
  LCD_ON, LCD_BLON: OUT STD_LOGIC);
END lcd_controller;

ARCHITECTURE FSMD OF lcd_controller IS
  TYPE state_type IS (s1,s2,s3,s4,s10,s11,s12,s13,s20,s21,s22,s23,s24);
  SIGNAL state: state_type;

  SUBTYPE ascii IS STD_LOGIC_VECTOR(7 DOWNTO 0);
  TYPE charArray IS array(0 to 15) OF ascii;
  TYPE initArray IS array(0 to 5) OF ascii;
  CONSTANT initcode: initArray := (x"38",x"38",x"38",x"0E",x"06",x"01");
```

```
  -- HELLO  WORLD
  CONSTANT line1: charArray := (x"20",x"20",x"48",x"45",x"4C",x"4C",x"4F",
                 x"20",x"20",x"57",x"4F",x"52",x"4C",x"44",x"20",x"20");
  SIGNAL count: INTEGER;

BEGIN

  LCD_ON <= '1';
  LCD_BLON <= '0';

  lcd_control: PROCESS(Clock, Reset)
  BEGIN
   IF(Reset = '1') THEN
     count <= 0;
     state <= s1;
   ELSIF(Clock'EVENT AND Clock = '1') THEN
     CASE state IS
     -- LCD initialization sequence
     -- The LCD_DATA is written to the LCD at the falling edge of the E line
     -- therefore we need to toggle the E line for each data write
     WHEN s1 =>
       LCD_DATA <= initcode(count);
       LCD_EN <= '1';   -- EN=1;
       LCD_RS <= '0';   -- RS=0; an instruction
       LCD_RW <= '0';   -- R/W'=0; write
       state <= s2;
     WHEN s2 =>
       LCD_EN <= '0';   -- set EN=0;
       count <= count + 1;
       IF count + 1 < 6 THEN
         state <= s1;
       ELSE
         state <= s10;
       END IF;

     -- move cursor to first line of display
     WHEN s10 =>
       LCD_DATA <= x"80";  -- x80 is address of 1st position on first line
     --LCD_DATA <= x"BF";  -- xBF is address of 1st position on second line
       LCD_EN <= '1';   -- EN=1;
       LCD_RS <= '0';   -- RS=0; an instruction
       LCD_RW <= '0';   -- R/W'=0; write
       state <= s11;
     WHEN s11 =>
       LCD_EN <= '0';   -- EN=0; toggle EN
       count <= 0;
       state <= s12;

     -- write 1st line text
     WHEN s12 =>
       LCD_DATA <= line1(count);
       LCD_EN <= '1';   -- EN=1;
       LCD_RS <= '1';   -- RS=1; data
       LCD_RW <= '0';   -- R/W'=0; write
       state <= s13;
     WHEN s13 =>
       LCD_EN <= '0';   -- EN=0; toggle EN
```

```
        count <= count + 1;
        IF count + 1 < 16 THEN
          state <= s12;
        ELSE
          state <= s20;
        END IF;
      WHEN s20 =>
        state <= s20;
      WHEN OTHERS =>
        state <= s20;
      END CASE;
    END IF;
  END PROCESS;
END FSMD;
```

图 9.32 16×2 LCD 控制器的 VHDL 代码

9.7 VGA 显示器控制器

在这一小节中我们会设计和实现用来控制 VGA 显示器的微型控制器。这个控制器可以控制 VGA 显示器显示简单的动态图像。

9.7.1 工作原理

低分辨率的 VGA 显示器一般包含 640 列×480 行的图像元素，也叫作**像素**。高分辨率的显示器有更多的行与列，但是主要思想还是一样的。一个图片的显示是通过点亮或者熄灭每个独立的像素来实现的。尽管点亮一个像素看上去没有什么区别，但是当一大片像素被同时点亮的时候。那么这个像素组合就可以显示衣服图像。控制器不停地扫描整个屏幕，以非常快的速度来点亮或者熄灭像素点。尽管只有每次扫描只有一个像素点被点亮，但是因为控制器扫描的速度非常快，由于视觉残留的作用，你可以看到整幅图像。

图 9.33 显示了扫描从左上角的第 0 行第 0 列开始，然后一直右移直到结束为止。当扫描到最后，它开始下一行。当扫描进行到最右下角的那个像素点的时候，它再次回到最左上角的那个像素点。为了减小屏幕上的频闪，整个屏幕的扫描速度都必须在每秒 60 次及以上。在横向和纵向的重描中，所有像素点都被熄灭。

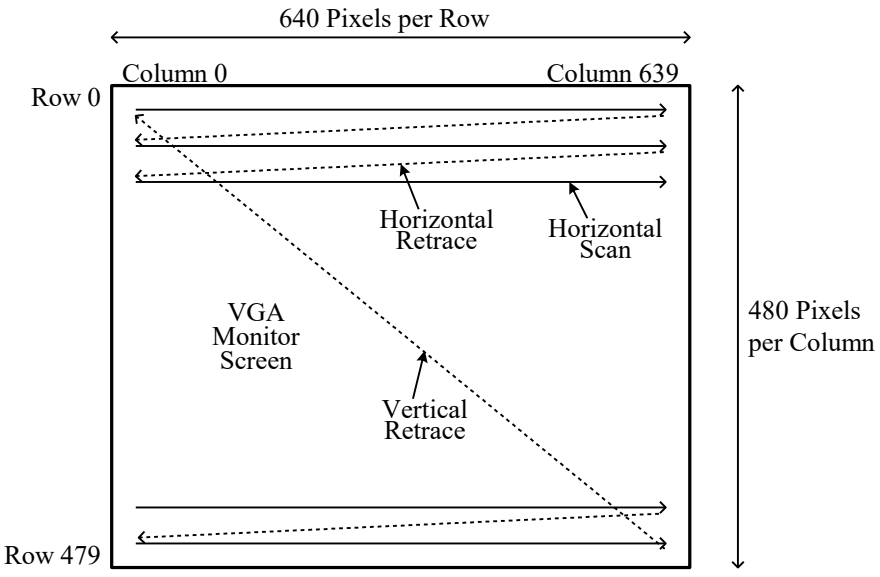


图 9.33 640 列×480 行的 VGA 显示器 扫描从第 0 行第 0 列开始不停往右往下倒第 639 行第 479 列